Rédiger un bon `README.md`

Guide général pour présenter et transmettre un projet

Création : 03/11/2026 · Version 1.0 · Mise à jour : 03/11/2026

Un `README` permet à une personne qui découvre votre projet de comprendre son utilité, de l'utiliser et de savoir ce qui fonctionne réellement. Il peut accompagner un logiciel, un jeu de données, une analyse, un rapport ou une collection de documents.

Répondre d'abord aux questions du lecteur

GitHub présente le `README` comme un point d'entrée : à quoi sert le projet, comment commencer, où trouver de l'aide et qui le maintient. Un fichier `README.md` placé à la racine du dépôt constitue un choix simple ; GitHub en affiche le contenu dans la page du dépôt. [1]

- Objet : que produit le projet, pour qui et pour quel besoin ?
- État actuel : qu'est-ce qui est disponible, incomplet ou encore prévu ?
- Utilisation : comment accéder au résultat ou reproduire un premier usage ?
- Vérification : comment contrôler le résultat, et avec quelles limites connues ?
- Repères : quels fichiers, sources et contacts permettent de poursuivre ?

Un document qui grandit avec le projet

Au début, un nom, un objectif et une indication sur l'état du travail peuvent suffire. Complétez le `README` lorsque les premiers fichiers et usages existent. Le plan de la page 3 est une réserve de rubriques, pas une liste à remplir intégralement.

Le `README` explique le projet à ses lecteurs. `AGENTS.md` donne à Codex des consignes durables sur la manière d'y travailler. La demande du moment reste dans le chat. OpenAI recommande `AGENTS.md` pour les conventions et instructions réutilisables. [3]

1. Donner des indications utilisables

Placez les informations dans l'ordre où une nouvelle personne en a besoin : comprendre, commencer, vérifier, puis approfondir. Expliquez un terme technique à sa première apparition.

Décrire ce qui existe

Distinguez les fonctions disponibles des idées à venir. Pour une analyse, précisez les données attendues et les sorties produites. Pour un rapport, indiquez sa question, son périmètre et le document à lire en premier. Pour un logiciel, décrivez un premier parcours d'utilisation.

Trop imprécis	**Plus utile pour le lecteur**
« Installation facile. »	Indiquer les étapes vérifiées, les prérequis et ce qui doit apparaître à la fin.
« Tout fonctionne. »	Décrire les usages essayés et les limites ; préciser ce qui n'a pas été vérifié.
« Voir les fichiers. »	Nommer les fichiers importants et expliquer à quoi chacun sert.

Rendre le démarrage reproductible

Si une installation est nécessaire, indiquez les prérequis vérifiés, le dossier de départ, les étapes dans l'ordre et le résultat attendu. Distinguez une adresse publique d'un service lancé sur l'ordinateur : une URL `localhost` désigne la machine du lecteur.

Ne recopiez pas une commande générique sans l'avoir essayée dans votre projet. S'il n'y a rien à installer, dites-le et indiquez comment ouvrir le résultat. Signalez les différences entre systèmes uniquement lorsqu'elles changent réellement les étapes.

Utiliser un peu de Markdown

`README.md` est un fichier texte. Markdown permet de structurer ce texte avec des titres, des listes, des liens et des blocs de commandes. Par exemple : # pour le titre principal, ## pour une rubrique, et - pour un élément de liste. [2]

Préférez des liens descriptifs aux mentions « cliquez ici ». Pour un autre fichier du dépôt, un lien relatif évite de dépendre du chemin personnel de votre ordinateur. Exemple : [Guide](`docs/guide.md`), uniquement si ce fichier existe à cet emplacement. [1, 2]

2. Un plan à copier et à adapter

Remplacez les indications entre crochets, puis supprimez les rubriques inutiles. Cet exemple est une proposition de rédaction, indépendante de tout projet particulier ; ce n'est pas un modèle officiel de GitHub ou d'OpenAI.

```
# [Nom du projet]

[En deux phrases : utilité, public et résultat produit.]

## État actuel
- Disponible : [fonctions ou livrables utilisables].
- Limites connues : [ce qui manque ou reste incertain].
- Prévu : [facultatif, clairement séparé de l'existant].

## Commencer
[Accès au résultat ou première étape d'utilisation.]
[Prérequis vérifiés, seulement si nécessaires.]
[Étapes exactes, dans l'ordre ; commandes si pertinentes.]

## Exemple d'utilisation
- Entrée ou action : [exemple concret].
- Résultat attendu : [ce que la personne doit obtenir].

## Vérifier
[Contrôles disponibles et manière de les réaliser.]
[Résultats observés, environnement et date si utiles.]
[Contrôles non réalisés ou limites de ces vérifications.]

## Se repérer
[Fichiers et dossiers importants, avec leur rôle.]
[Sources, données ou documentation complémentaires.]

## Aide et reprise
[Où signaler un problème ou poser une question.]
[Responsable ou équipe ; modalités de contribution si utiles.]
```

Selon le projet, ajoutez une courte rubrique sur la provenance des données, les conditions de réutilisation, la manière de citer le travail ou les choix de conception importants. Renvoyez vers un document séparé lorsque les explications deviennent longues.

Pour commencer petit : gardez seulement le titre, l'objectif, l'état actuel et « Commencer ». Une rubrique vide ou une affirmation non vérifiée aide moins qu'une limite clairement formulée.

3. Vérifier et tenir à jour

Faire le test d'une première lecture

1. Ouvrez le `README` comme si vous découvriez le projet. Son utilité et son état actuel sont-ils compréhensibles dès le début ?
2. Suivez les étapes de démarrage depuis le point de départ annoncé, sans utiliser une étape connue de vous seul. Comparez le résultat obtenu au résultat décrit.
3. Vérifiez les liens, les noms de fichiers et les commandes. Contrôlez le rendu Markdown dans l'outil utilisé pour lire le fichier.
4. Faites essayer le parcours à une autre personne si possible. Corrigez les passages qui l'obligent à deviner ; indiquez ce qui n'a pas pu être essayé.

S'aider de Codex sans inventer

Demandez à Codex de s'appuyer sur les fichiers et sur des résultats observés. Un texte bien rédigé ne suffit pas à prouver qu'une commande fonctionne. OpenAI conseille de définir le résultat attendu et de vérifier le travail avant de l'accepter. [3]

> *« Examine les fichiers utiles et propose une mise à jour du `README`. Décris uniquement les fonctions présentes. Vérifie les étapes de démarrage et les commandes que tu peux exécuter. Signale celles que tu ne peux pas essayer. N'invente ni lien, ni résultat de test, ni prérequis. Conserve les ajouts existants et indique les informations manquantes. »*

Relisez le résultat et gardez la décision finale. Actualisez le `README` lorsqu'un changement modifie l'usage, les prérequis, les sorties ou les limites. L'historique détaillé peut rester dans Git ou dans un journal de versions ; le `README` doit permettre de comprendre la version consultée.

Sources officielles

[1] GitHub — About the repository `README` file

[2] GitHub — Basic writing and formatting syntax

[3] OpenAI — Best practices

Sources consultées le 03/11/2026. La méthode, les reformulations et le plan à copier sont une synthèse pédagogique originale. Adaptez-les à l'usage réel du projet.